E Waste Compute Grid
Team Development Plan

Owners milestones and integration gates

Prepared for Kushagra, Vaibhav, Nikhil and Chinmay
Planning baseline 7 October 2026

Our first shared milestone

We will make one real image job work from the dashboard through one physical worker and back to a downloadable result. Then we will expand to three machines, demonstrate worker failure and recovery, and deploy the same verified flow through AWS.

Each member owns a component, but a phase is complete only when the components work together. This document assigns work; it does not claim the proposed changes are already implemented.

Primary ownership

Member	Owns	Works closely with
Kushagra	Control plane, scheduler, task lifecycle, database and API contracts	Vaibhav on reliability and AWS; Nikhil on scheduling inputs
Vaibhav	Worker agent, Docker execution, networking, runtime reliability and deployment	Kushagra on worker protocol; Nikhil on workload runtime
Nikhil	AI workloads, genuine datasets, model/runtime validation, benchmarks and energy methodology	Vaibhav on execution; Kushagra on telemetry and scoring
Chinmay	React dashboard, API integration, readable results and browser verification	Kushagra on read APIs; all members on demo evidence

What we can reuse

The existing pipeline includes registration, heartbeats, job splitting, PostgreSQL task claims, inference, uploads, aggregation, leases and retries. The earlier audit passed 38 tests and application builds. Its workers ran locally in native mode; browser behavior, three physical devices, Docker edge execution and AWS deployment remain unverified.

Architecture decision Keep PostgreSQL as the queue and use outbound worker polling. Do not add Redis, SQS or a new framework before a demonstrated need.

Current blockers Copied Python environment, inconsistent local database settings, synthetic sample photos, missing task ownership/history views and advanced utilities not connected to the live system.

Start together with these four assignments

Begin these preparation tasks together. Integrate as soon as the backend and worker are reachable; do not wait for all components to become polished. There is no fixed deadline until the team confirms device access and availability.

Kushagra establish the integration contract

Verify the local database and API topology. Document the existing upload, job, claim, heartbeat and completion bodies, including authentication, idempotency keys and lease tokens. Strengthen result validation to account for the exact assigned image identities.

Files control-plane/src/config.ts, app.ts, validation.ts; prisma/schema.prisma; docs/api_contracts.md; control-plane/tests.

Deliver A reachable backend, tested request examples and a result contract reviewed by Vaibhav, Nikhil and Chinmay.

Vaibhav prepare the first real worker

Create a fresh supported Python environment. Build or load both approved workload images, verify Docker and paths on one physical machine, and configure its reachable backend address. Keep the worker identity private and distinct.

Files worker/config.py, api_client.py, task_executor.py, README.md, .env.example; infra/docker-compose.yml and Dockerfiles.

Deliver A worker that registers, verifies its real runtime, downloads inputs, executes and returns results without manual task operation.

Nikhil replace placeholders with useful inputs

Prepare ten genuine photographs with expected labels and provenance. Make download failure visible instead of silently saving blank images. Verify model/labels, record real predictions, and explain errors or low confidence honestly.

Files workloads/test-data/setup_sample_data.py and sample data; ai-inference/run_inference.py; model/runtime documentation.

Deliver A genuine input manifest and valid result examples handed to Kushagra and Chinmay; runtime instructions handed to Vaibhav.

Chinmay verify the user journey

Connect the dashboard to the real backend. Clearly distinguish the pipeline demo from image classification. Verify login, upload, submission, live progress and result download; show meaningful connection and submission errors.

Files frontend/src/App.tsx, services/api.ts, App.css; frontend/vite.config.ts; browser verification notes.

Deliver A repeatable browser flow using Nikhil's photos and Vaibhav's worker, with screenshots and sanitized job IDs.

Shared acceptance Ten real images, chunk size five, two tasks, one physical worker, every input counted once as a prediction or explicit error, and a downloaded result. Relevant checks must run with database integration enabled.

Phases 0 to 3 establish the working slice

These phase numbers match the master project plan. Keep working registration and queue code; fix demonstrated defects and add the missing evidence.

Phase 0 Recover the reproducible baseline

Kushagra Recover the canonical Git checkout; choose database topology; document backend startup and environment variables.

Vaibhav Recreate worker setup; verify Docker/images, identity paths, power, networking and sleep settings.

Nikhil Verify model/labels/dependencies and separate real demonstration data from synthetic fixtures.

Chinmay Verify frontend build and API proxy; document how a teammate connects.

Integration gate A teammate can start the backend and one registered worker from clean instructions without the copied virtual environment.

Phase 1 Validate real inputs and contracts

Kushagra Own API examples and exact assigned-input result validation; add meaningful backend regression tests.

Vaibhav Verify workload capability before advertising it; align worker request and reporting bodies with the contract.

Nikhil Curate labeled real photos; remove silent realistic-looking fallbacks; validate classifier output and schema.

Chinmay Make workload modes clear; align upload/job/result types and user-visible validation errors.

Integration gate Real inputs produce valid predictions or explicit errors; clients share the same reviewed contract.

Phase 2 Complete the first browser and worker flow

Kushagra Trace upload, creation, claim, acceptance and aggregation; retain task/job IDs and check counters.

Vaibhav Run the actual approved container on a physical device; verify staging, upload, cleanup and heartbeat.

Nikhil Confirm the actual model output on the submitted ten-photo dataset and distinguish accuracy from delivery.

Chinmay Drive and record dashboard submission, progress and result download against the real backend.

Integration gate The ten-photo two-task acceptance run passes and another teammate can repeat it.

Phase 3 Add task and assignment history

Kushagra Design and migrate TaskAttempt history and timing; expose bounded authenticated job/task/attempt read APIs.

Vaibhav Report agreed phase/timing metadata and verify lease/attempt handling across retries and reconnects.

Nikhil Validate original image identity and timing/metric meaning; provide readable result examples.

Chinmay Build job detail, task-owner/retry timeline and prediction table using the new APIs.

Integration gate Who ran each task and what happened to every attempt is visible and survives a control-plane restart.

Review responsibility Kushagra leads phase integration; Vaibhav reviews worker-facing APIs, Chinmay reviews dashboard-facing APIs, and Nikhil reviews workload/result semantics. Schema changes require migration and concurrency review.

Phases 4 to 8 prove the hackathon system

Phase 4 Run three machines and recover a failure

Kushagra Verify compatible concurrent claims, expired-lease requeue and rejection of late results.

Vaibhav Prepare three distinct physical identities; abruptly kill an assigned worker and verify restart cleanup.

Nikhil Choose enough genuine work to show overlap; record hardware and runtime conditions.

Chinmay Show the lost worker, affected task, new owner/attempt and final result.

Integration gate Three physical machines contribute; unfinished work is recovered elsewhere with one accepted result.

Phase 5 Report performance and energy honestly

Kushagra Store agreed attempt timings/provenance; separate job elapsed time from summed task durations.

Vaibhav Connect Nikhil's measurement adapter to execution; collect agreed runtime metadata.

Nikhil Own baseline calibration, optional actual meter, throughput experiment and measurement methodology.

Chinmay Display job/node contribution, throughput and energy with units, scope and estimated/measured labels.

Integration gate Numbers have reproducible methods; excluded costs and uncertainties are stated; no assumed cloud savings.

Phase 6 Protect the service before public exposure

Kushagra Own authorization, result validation, credential lifecycle, limits and safe API errors.

Vaibhav Own worker credential storage, runtime isolation, endpoint checks and TLS/network setup.

Nikhil Verify workload/model provenance and safe input decoding/resource requirements.

Chinmay Handle unauthorized/degraded states; keep secrets out of frontend builds and shared evidence.

Integration gate Public deployment uses unique credentials, HTTPS, scoped access and private data services.

Phase 7 Make failures diagnosable

Kushagra Add correlated structured logs, readiness/liveness, backlog and recovery-sweep metrics.

Vaibhav Collect sanitized worker/container logs and configure retention and useful alerts.

Nikhil Validate throughput/energy definitions and identify measurement coverage limits.

Chinmay Expose fresh/stale state and understandable errors; verify failure information is visible.

Integration gate A teammate can diagnose no-worker, corrupt-input and storage-failure stalls from retained evidence.

Phase 8 Deploy and repeat through AWS

Kushagra Own backend/database migration and API/storage/IAM architecture; review deployment security.

Vaibhav Lead deployment scripts, HTTPS, networking, secret injection, logs, rollback and teardown.

Nikhil Validate the model/runtime on cloud-connected physical workers and real S3 transfers.

Chinmay Serve the built dashboard properly and verify the complete deployed browser flow.

Integration gate The same real workload and recovery demonstration passes through deployed HTTPS and actual S3.

Phases 9 to 12 strengthen the portfolio

The hackathon release can freeze after phases 0 to 8 pass. Do not wait for every advanced feature. Portfolio work builds on the verified release without destabilizing it.

Phase 9 Make scheduling explainable

Kushagra Implement fresh load/benchmark eligibility and a live scheduling policy; persist inputs and rationale.

Vaibhav Report bounded fresh CPU/RAM/runtime telemetry and enforce local refusal when capacity is insufficient.

Nikhil Measure per-workload throughput under real resource limits; validate policy inputs and comparisons.

Chinmay Show actual eligibility and selection rationale, with measurement timestamp and policy version.

Integration gate The policy affects live decisions and is compared with FIFO; fixture scores are not presented as live scheduling.

Phase 10 Improve operations and maintainability

Kushagra Extract backend responsibilities; own reviewed migrations, constraints, admission and optional cancellation.

Vaibhav Add service install/start/stop, drain/reconnect and cleanup; rehearse backup/restore and deployments.

Nikhil Verify runtime compatibility and repeatable measurement across supported devices.

Chinmay Test management controls, pagination, keyboard/accessibility and browser error paths.

Integration gate Clean setup, relevant CI, Docker/browser/fault tests, five-worker soak and restore/rollback evidence pass.

Phase 11 Add a second workload and optional energy policy

Kushagra Define versioned splitter/validator/aggregator adapters; preserve common lease/retry behavior.

Vaibhav Integrate the approved workload runtime and capability advertisement without arbitrary code execution.

Nikhil Build a genuine second independent workload; supply trusted benchmark/energy inputs for any optimizer.

Chinmay Add workload-specific input/results views and explain any enabled policy.

Integration gate Two real workloads share orchestration; energy optimization is claimed only when measured and used live.

Phase 12 Freeze and publish the verified release

Kushagra Freeze contracts and tag the canonical release; finalize architecture, decisions, API and limits.

Vaibhav Finalize preflight/startup/deployment instructions and rehearse actual worker failure.

Nikhil Prepare reproducible dataset/benchmark/energy evidence and the hardware reuse explanation.

Chinmay Lead the 3–5 minute dashboard demonstration, screenshots and recorded fallback.

Integration gate Demo and portfolio claims match evidence; no untested feature is presented as complete.

Work we deliberately defer

Redis/SQS, global worker reservations, adaptive chunking, live push updates and public volunteer execution require a demonstrated need and separate acceptance criteria. Image classification is the first supported real workload; a second workload should follow a stable core release.

Handoffs and shared interfaces

Agree on these deliverables before dependent implementation. Share a validated example and executed check, not only a screenshot or verbal description.

From	To	Handoff
Kushagra	Vaibhav	Registration, heartbeat, claim, upload and complete/fail bodies; auth; lease/retry semantics; test API URL.
Kushagra	Chinmay	Typed job/node/task/attempt/results responses; pagination, error cases and metric definitions.
Nikhil	Vaibhav	Approved image/tag or digest; command; input/output paths; model/labels; resource requirements and runtime preflight.
Nikhil	Kushagra	Input identity/result schema; genuine data; versioned throughput and energy source/scope.
Nikhil	Chinmay	Expected prediction/error examples, interpretation of confidence and measurement labels.
Vaibhav	All	Clean setup/deploy commands; reachable addresses; device inventory, release version and sanitized runtime evidence.
Chinmay	All	Reproducible browser/demo script, screenshots and failures found at the API integration boundary.

File ownership prevents conflicting changes

Kushagra owns control-plane source, Prisma and shared API contracts. Vaibhav owns worker orchestration/execution and infra. Nikhil owns workload scripts, datasets, benchmarks and energy utilities. Chinmay owns frontend source and browser tests. Work on another owner's file through a reviewed change; coordinate before simultaneous edits.

Important boundary Nikhil owns measurement logic; Vaibhav connects it to worker execution; Kushagra stores and aggregates it; Chinmay displays it. No member independently invents a different energy schema.

Contract changes and review

Document changed requests/responses, statuses, units and version impact before merging. The primary owner proposes the change; every affected client owner reviews it. Keep compatibility where reasonable and add a regression test for any behavior change.

Required reviewers Worker protocol: Kushagra and Vaibhav. Workload output: Nikhil and Kushagra. Dashboard reads: Kushagra and Chinmay. Runtime packaging: Vaibhav and Nikhil. Deployment/security: Kushagra and Vaibhav.

Data rules Never share tokens, presigned URLs, credential directories or database dumps in team updates. Synthetic results are labeled. Unknown energy is not zero energy. Repeated execution may happen; accepted results must not be counted twice.

Integration workflow and release checklist

Daily working rhythm

Start with a short check-in: what changed, what is blocked, and which shared contract is affected. Work in small reviewable branches in the canonical repository. Integrate toward the current slice as soon as a component is usable. End with a short live run or executed check and an update to the evidence ledger.

Status update format Member; task/phase; changed files; contract/schema impact; checks executed and environment; result; blocker; next step. Label unexecuted checks explicitly.

Done means Implementation, relevant validation, an integration demonstration, current documentation and review by an affected owner. A file existing or a test using mocks is not sufficient evidence of a real feature.

Who verifies what

Owner	Primary checks	Integration partner
Kushagra	API/database/scheduler concurrency, idempotency, leases, exact result validation and migrations	Vaibhav for protocol; Nikhil for result semantics
Vaibhav	Real Docker execution, paths/limits, reconnect, process kill, cleanup, networking and deployment	Kushagra for recovery; Nikhil for runtime
Nikhil	Genuine data, inference correctness, corrupt-input handling, benchmark and energy methods	Vaibhav for actual execution
Chinmay	Frontend build/lint, browser workflow, progress/results, errors, accessibility and demo	Kushagra for API consistency

Backend integration requires TEST_DATABASE_URL and a working TEST_PYTHON when the default environment is unavailable. Use a disposable database. Record the actual suite and platform; a successful command can still have skipped database integration.

Hackathon release gate

- Three distinct authorized physical machines contribute to one genuine job with overlapping execution.
- An assigned worker is abruptly killed; another machine finishes its task under a new attempt.
- Every input is accounted for exactly once in accepted results; failures and partial success are explicit.
- Dashboard shows task ownership/retry and readable output, with correct timing and energy labels.
- Clean setup and relevant checks pass; AWS is demonstrated only if real deployed transfers were verified.
- A rehearsed 3–5 minute demonstration and clearly labeled fallback match the frozen release.

Portfolio follow through

Retain reviewed code, migrations, CI/browser/Docker/fault evidence, restore/rollback instructions, architecture/API documentation, same-work benchmark methods, screenshots/video and known limits. Keep the master plan evidence ledger current after every completed phase.

Next team checkpoint Run the ten-photo two-task browser-to-physical-worker slice together. Kushagra coordinates the gate; Vaibhav operates the worker, Nikhil verifies outputs and Chinmay drives the dashboard.